FINGERPRINT ENROLLMENT APP

Technical Project Brief & Debugging Handoff

Revision 1.1 — 2026-10-13. Prepared from the supplied fingerprint-app.rar source archive; revision 1.1 folds in the findings, evidence and code changes of the 502/503 debugging session of 13 Oct 2026.

Purpose. This document gives another AI assistant enough architectural context, implementation details, important source snippets, and known limitations to help diagnose bugs in this project. It is a technical handoff, not a claim that the current project has been fully tested on the target machine.

Project in one sentence. A local TypeScript application for enrolling people with personal details and fingerprint templates, using a React/Vite frontend, an Express API with Prisma and SQLite, and a local scanner bridge intended to communicate with a Suprema BioMiniWebAgent.

Revision history

Rev	Date	Changes
1.0	—	Original handoff generated from the source archive.
1.1	2026-10-13	§7: confirmed BioMiniWebAgent response quirk (retValue returned as string "0"; getScannerStatus may omit retValu

Layer overview

Layer	Technology / responsibility
Frontend	React 18, TypeScript, Vite, Persian RTL UI, port 5173
API	Node.js, Express, Zod validation, Prisma ORM, port 3000
Database	SQLite by default; Prisma schema can be adapted to PostgreSQL
Scanner agent	Express service on 127.0.0.1:8443; mock mode or BioMini bridge
Biometric security	AES-256-GCM encryption of templates before database storage
Matching	Mock exact-byte matcher or planned Suprema UFMatcher integration

1. Repository Layout

```
fingerprint-app/
  package.json                # npm workspaces and top-level scripts
  api/
    src/server.ts             # enrollment API and duplicate checks
    src/crypto.ts             # AES-256-GCM encrypt/decrypt
    src/matcher.ts            # mock or agent-based fingerprint matching
    src/normalize.ts          # Persian/Arabic digit normalization
    prisma/schema.prisma      # Person model and SQLite config
    tsconfig.json             # MUST include only ["src"] (fixed in rev 1.1)
    .env.example              # safe configuration template
  agent/
    src/server.ts             # mock routes and real-mode route wiring
    src/biomini.ts            # HTTPS bridge to BioMiniWebAgent (patched rev 1.1)
    .env.example
  web/
    src/main.tsx              # React entry point
    src/EnrollForm.tsx        # enrollment form and submit/scan flow
    src/PeopleList.tsx        # registered-person list
    src/scanner.ts            # frontend-to-agent API adapter
```

```
    src/styles.css
    .env.example
  README.md
```

The archive also contains dependency/build artifacts (Vite cache files, a Prisma cache/database). They are not the source of truth for debugging. Do not share real .env files or database contents with an external AI unless sanitized.

2. Runtime Architecture

```
Browser / React (http://localhost:5173)
  |-- GET  http://localhost:8443/api/status
  |-- POST http://localhost:8443/api/capture
  |       -> scanner agent -> BioMiniWebAgent HTTPS 127.0.0.1:8083
  |
  |-- GET  /api/people       (Vite proxy expected)
  |-- POST /api/people       (Vite proxy expected)
          -> Express API 127.0.0.1:3000
              -> validate input with Zod
              -> normalize national ID / phone digits
              -> check duplicate national ID
              -> decrypt enrolled templates in memory
              -> run matcher
              -> encrypt new template with AES-256-GCM
              -> save with Prisma / SQLite
              -> read back and verify decryption integrity
```

Integration detail. The frontend calls the API using relative paths such as /api/people, while the scanner agent uses an explicit localhost URL. The Vite proxy in web/vite.config.ts must route /api to the API port, or frontend API calls hit the Vite server and fail.

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: { "/api": "http://localhost:3000" },
  },
});
```

2.1 Error-status map (rev 1.1)

Status / signal	Where	Meaning
502	agent	BiominiError with default status: initDevice found no scanner, captureSingle / getTemplateData failure, or
503	agent	Connection-level failure to BioMiniWebAgent: ECONNREFUSED, TLS handshake error or TIMEOUT (Bi
503	API	Matcher failure: with MATCHER=agent the real /api/match returns 501 (known limitation, §6).
409 / 400 / 500	API	Duplicate national ID or fingerprint / invalid input / save or integrity failure (§4).
curl: (7), browser ERR_CONNECTION_REFUSED	any	Nothing listening: the service process itself is down (or bound to another address family — test localho

3. Install and Run

The README specifies Node.js 18 or newer (session machine ran Node v24.21.0). From the repository root:

```
npm install
# Create api/.env from api/.env.example and set FP_ENC_KEY:
```

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
npm run db:setup
npm run dev
# Open http://localhost:5173
```

To test repeatable mock scans, copy web/.env.example to web/.env and set VITE_MOCK_FINGER=alice, then restart the frontend. Enrolling the same mock finger twice should trigger the mock exact-template duplicate check. This test does not prove real biometric matching works.

Root scripts: npm run dev launches agent, API and frontend concurrently; if one workspace process dies or its window is closed the others can go down with it — treat a sudden curl: (7) on port 8443 as a possible whole-stack exit. npm run typecheck runs TypeScript checks for API, agent and web (in rev 1.0 it failed at the api workspace because of the tsconfig bug fixed in rev 1.1, §10). npm run db:setup generates the Prisma Client and runs prisma db push.

4. API Behavior and Enrollment Logic

POST /api/people accepts fullName, optional nationalId, optional phone, fingerprint (base64) and fpFormat. It returns 201 with an id when saved; 400 for invalid input; 409 for a duplicate national ID or fingerprint; 503 when the matcher service fails; 500 for an unexpected save/integrity failure. Error bodies are Persian UI sentences (rev 1.0 printed them as black squares; meanings: "name is required", "fingerprint is required", "this national ID is already enrolled", "this fingerprint is already enrolled", "matcher service unavailable, try again", "storage integrity check failed, record rolled back", "unexpected API error"). GET /api/people returns public person fields only; it intentionally excludes encrypted fingerprint bytes.

```
const enrollSchema = z.object({
  fullName: z.string().trim().min(1, /* Persian: name required */ "...").max(200),
  nationalId: z.string().trim().max(50).optional().or(z.literal("")),
  phone: z.string().trim().max(30).optional().or(z.literal("")),
  fingerprint: z.string().min(1, /* Persian: fingerprint required */ "..."), // base64
  fpFormat: z.string().min(1).max(50),
});

// Runs enrollments one at a time so two simultaneous requests
// cannot both pass the duplicate check and both insert.
let queue: Promise<unknown> = Promise.resolve();
function serialize<T>(fn: () => Promise<T>): Promise<T> {
  const run = queue.then(fn, fn);
  queue = run.catch(() => undefined);
  return run;
}
```

Duplicate check and storage flow (condensed from api/src/server.ts):

```
try {
  const result = await serialize(async () => {
    if (nationalId) {
      const existing = await prisma.person.findUnique({ where: { nationalId } });
      if (existing) return { status: 409 as const, error: /* Persian: ID enrolled */ "..." };
    }
    const rows = await prisma.person.findMany({ select: { id: true, fingerprint: true } });
    const candidates = rows.map((r) => ({
      id: r.id,
      template: decrypt(Buffer.from(r.fingerprint)),
    }));
```

```
    let matchedId: number | null;
    try {
      matchedId = await matcher.findMatch(probe, candidates);
    } catch (err) {
      console.error("[matcher]", err);
      return { status: 503 as const, error: /* Persian: matcher unavailable */ "..." };
    }
    if (matchedId !== null)
      return { status: 409 as const, error: /* Persian: finger enrolled */ "..." };
    const person = await prisma.person.create({
      data: { fullName, nationalId, phone, fpFormat, fingerprint: encrypt(probe) },
      select: { id: true },
    });
    // Read the row back and decrypt it: proves what is in the database
    // is exactly what the scanner produced.
    const saved = await prisma.person.findUnique({
      where: { id: person.id }, select: { fingerprint: true },
    });
    const intact = saved !== null && decrypt(Buffer.from(saved.fingerprint)).equals(probe);
    if (!intact) {
      await prisma.person.delete({ where: { id: person.id } });
      return { status: 500 as const, error: /* Persian: integrity failed */ "..." };
    }
    return { status: 201 as const, id: person.id };
  });
  if (result.status !== 201) return res.status(result.status).json({ error: result.error });
  return res.status(201).json({ id: result.id });
} catch (err) {
  if (err instanceof Prisma.PrismaClientKnownRequestError && err.code === "P2002")
    return res.status(409).json({ error: /* Persian: ID enrolled */ "..." });
  console.error(err);
  return res.status(500).json({ error: /* Persian: unexpected API error */ "..." });
}
```

5. Fingerprint Encryption

```
import crypto from "node:crypto";
function getKey(): Buffer {
  const hex = process.env.FP_ENC_KEY;
  if (!hex || !/^[0-9a-fA-F]{64}$/.test(hex)) {
    throw new Error("FP_ENC_KEY must be set to 64 hex characters (32 bytes). See .env.example");
  }
  return Buffer.from(hex, "hex");
}
/** AES-256-GCM. Output layout: iv(12) | authTag(16) | ciphertext */
export function encrypt(plain: Buffer): Buffer {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv("aes-256-gcm", getKey(), iv);
  const data = Buffer.concat([cipher.update(plain), cipher.final()]);
  return Buffer.concat([iv, cipher.getAuthTag(), data]);
```

```
}
export function decrypt(blob: Buffer): Buffer {
  const iv = blob.subarray(0, 12);
  const tag = blob.subarray(12, 28);
  const data = blob.subarray(28);
  const decipher = crypto.createDecipheriv("aes-256-gcm", getKey(), iv);
  decipher.setAuthTag(tag);
  return Buffer.concat([decipher.update(data), decipher.final()]);
}
/** Fails fast at startup if the key is missing or malformed. */
export function assertKey(): void { getKey(); }
```

Stored byte layout is IV (12 bytes) + authentication tag (16 bytes) + ciphertext. FP_ENC_KEY must be exactly 64 hexadecimal characters. Losing the key makes stored templates unrecoverable; a key leak compromises confidentiality. For production use a secrets manager or OS keystore and secure backups.

6. Matching Strategy

```
export interface Candidate { id: number; template: Buffer; }
export interface Matcher {
  /** Returns the id of an already-enrolled person whose fingerprint matches, or null. */
  findMatch(probe: Buffer, candidates: Candidate[]): Promise<number | null>;
}
/** Development only. Real scans of the same finger are never byte-identical. */
class MockMatcher implements Matcher {
  async findMatch(probe: Buffer, candidates: Candidate[]): Promise<number | null> {
    const hit = candidates.find((c) => c.template.equals(probe));
    return hit ? hit.id : null;
  }
}
/** Production matcher: delegates to the local agent (Suprema UFMatcher expected). */
class AgentMatcher implements Matcher {
  constructor(private readonly baseUrl: string) {}
  async findMatch(probe: Buffer, candidates: Candidate[]): Promise<number | null> {
    if (candidates.length === 0) return null;
    const res = await fetch(`${this.baseUrl}/api/match`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        probe: probe.toString("base64"),
        candidates: candidates.map((c) => ({ id: c.id, template: c.template.toString("base64") })),
      }),
    });
    if (!res.ok) throw new Error(`Matcher agent error: HTTP ${res.status}`);
    const data = (await res.json()) as { matchedId: number | null };
    return data.matchedId ?? null;
  }
}
export function createMatcher(): Matcher {
  if (process.env.MATCHER === "agent") {
    return new AgentMatcher(process.env.AGENT_URL ?? "http://localhost:8443");
```

```
  }
  console.warn("[matcher] MATCHER=mock: duplicate detection is exact-match only. Do not use in
production.");
  return new MockMatcher();
}
```

The default mock matcher compares decrypted template bytes exactly — deterministic software tests only. Two legitimate scans of the same finger are not normally byte-identical. Real duplicate detection requires a genuine biometric matcher (Suprema UFMatcher) and a properly implemented agent /api/match endpoint, which currently returns 501 in real mode.

7. Scanner Agent and Suprema Bridge

agent/src/server.ts exposes /api/status, /api/capture and /api/match. With AGENT_MODE=biomini the real-mode routes are registered before the mock routes; the real capture path delegates to agent/src/biomini.ts. The real-mode /api/match route currently returns HTTP 501, so production duplicate matching is incomplete.

```
GET  /api/status   -> { mode: "mock" | "device", scannerConnected: boolean, model?: string }
POST /api/capture  -> { template: "<base64>", format: "SUPREMA" | "MOCK" }
POST /api/match    <- { probe: "<base64>", candidates: [{ id, template }] }
                   -> { matchedId: number | null }
```

7.1 Bridge facts

The bridge talks to BioMiniWebAgent over HTTPS on 127.0.0.1:8083 with a legacy TLS agent (rejectUnauthorized: false, minVersion TLSv1, ciphers ALL:@SECLEVEL=0, SSL_OP_LEGACY_SERVER_CONNECT) because the vendor service ships a self-signed certificate and an old TLS stack. It caches a session cookie (username=<sessionId> from /api/createSessionID) and the current scanner handle in module state; doCapture() resets the handle (scanner = null) on any error so the next attempt re-initialises. This is a compatibility workaround, not a safe pattern for general network traffic.

7.2 HTTP helper (unchanged)

```
function call(path: string, params: Record<string, string | number> = {}, timeoutMs = 15000):
Promise<Reply> {
  const qs = new URLSearchParams({ dummy: String(Math.random()) });
  for (const [k, v] of Object.entries(params)) qs.set(k, String(v));
  return new Promise((resolve, reject) => {
    const req = https.request({
      host: HOST, port: PORT, path: `${path}?${qs}`, method: "GET", agent: tlsAgent,
      headers: cookie ? { Cookie: cookie } : {}, timeout: timeoutMs,
    }, (res) => { /* collect body; JSON.parse(body.replace(/^\uFEFF/, "")) */ });
    req.on("timeout", () => req.destroy(new Error("TIMEOUT")));
    req.on("error", (err) => reject(new BiominiError(/* Persian: vendor unreachable */ "...", 503)));
    req.end();
  });
}
```

7.3 Vendor response quirk — CONFIRMED 2026-10-13 (new in rev 1.1)

The installed BioMiniWebAgent build returns **retValue as a string**, and its getScannerStatus reply can omit retValue entirely. Evidence captured from GET /api/debug on the session machine:

```
{"target":"https://127.0.0.1:8083","node":"v24.21.0",
 "createSessionID":{"httpStatus":200,"body":"{ "sessionId" : "2026913114035_2130706433" }"},
 "initDevice":{"httpStatus":200,"body":"{
    "ScannerCount" : 1,
```

```
    "ScannerInfos" : [ { "DeviceHandle" : 24208216, "DeviceIndex" : 0,
                         "DeviceType" : "Error", "ScannerName" : "<mojibake>" } ],
    "retString" : "Success",
    "retValue" : "0" }"},          <-- STRING, not the number 0
 "getScannerStatus":{"httpStatus":200,"body":"{ "sessionId" : "0" }"}   <-- no retValue at all
}
```

Consequence in rev 1.0 code: the strict test `r.retValue === 0` is **false** for the string "0" (JavaScript strict equality), so initScanner() discarded a healthy scanner, /api/status reported scannerConnected:false, and every capture failed with the scanner-not-found BiominiError (HTTP 502 at the agent). The same strict comparisons would also break captureSingle, getTemplateData and getScannerStatus. Secondary observations, still open: DeviceType:"Error" and a mojibake ScannerName (an encoding artifact — display only, it feeds the status model field); getScannerStatus without retValue.

Exact Persian message produced by that path (restored from rev 1.0's black squares):

اسکنر پیدا نشد. کابل USB را بررسی کنید و برنامه‌های دیگری که از اسکنر استفاده می‌کنند را ببندید.

7.4 Patched bridge code (rev 1.1) — agent/src/biomini.ts

```
/** Vendor builds return retValue inconsistently as number 0 or string "0".
 *  Normalize before comparing; never use === 0 on it directly. */
function retOk(reply: Reply): boolean {
  const v = (reply as { retValue?: unknown }).retValue;
  if (v === undefined || v === null) return false;
  return Number(v) === 0;
}

async function initScanner(): Promise<ScannerInfo | null> {
  await ensureSession();
  const r = await call("/api/initDevice");
  const list = (r.ScannerInfos ?? []) as ScannerInfo[];
  scanner = retOk(r) && list.length > 0 ? list[0] : null;   // was: r.retValue === 0
  if (!scanner)
    console.error("[biomini] initDevice found no scanner. Reply:", JSON.stringify(r).slice(0, 300));
  return scanner;
}

export async function biominiStatus(): Promise<{ mode: "device"; scannerConnected: boolean; model?:
string }> {
  if (!scanner) await initScanner();
  if (!scanner) return { mode: "device", scannerConnected: false };
  const s = await call("/api/getScannerStatus", { sHandle: scanner.DeviceHandle });
  // Lenient: this build's getScannerStatus may omit retValue; fail only on an explicit non-zero.
  const v = (s as { retValue?: unknown }).retValue;
  if (v !== undefined && Number(v) !== 0) {
    console.error("[biomini] getScannerStatus failed:", JSON.stringify(s).slice(0, 300));
    scanner = null;
    return { mode: "device", scannerConnected: false };
  }
  return { mode: "device", scannerConnected: true, model: scanner.ScannerName };
}
async function doCapture(): Promise<{ template: string; format: string }> {
```

```
  try {
    if (!scanner) await initScanner();
    if (!scanner) {
      throw new BiominiError(
"اسکنر پیدا نشد. کابل USB را بررسی کنید و برنامه‌های دیگری که از اسکنر استفاده می‌کنند را ببندید".
      );
    }
    const handle = scanner.DeviceHandle;
    const cap = await call(
      "/api/captureSingle",
      { sHandle: handle, id: pageId, resetTimer: 30000 },
      45000
    );
    if (!retOk(cap))                                    // was: cap.retValue !== 0
      throw new BiominiError(
"}`خطای نامشخص: ${cap.retString ?? "اسکن انجام نشد`
      );
    const tpl = await call("/api/getTemplateData", {
      sHandle: handle, id: pageId, encrypt: 0, encryptKey: "", extractEx: 0,
      qualityLevel: QUALITY_LEVEL,
    });
    const b64 = typeof tpl.templateBase64 === "string"
      ? tpl.templateBase64.replace(/\s+/g, "") : "";
    if (!retOk(tpl) || Buffer.from(b64, "base64").length < 32) {   // was: tpl.retValue !== 0
      throw new BiominiError(
.`استخراج اثر انگشت انجام نشد )کیفیت پایین؟.${tpl.retString ?? ""}( دوباره اسکن کنید`
      );
    }
    return { template: b64, format: "SUPREMA" };
  } catch (err) {
    scanner = null; // re-initialise on the next attempt
    throw err;
  }
}
```

If the Reply interface types retValue as number, widen it to `retValue?: number | string` so npm run typecheck stays green. Why the fix works: Number("0") === 0 is true, so success replies are recognised regardless of the vendor build's typing; failure replies (non-zero retValue plus retString) still throw with the vendor's own message. Regression risk: low — a build that signals success by omitting retValue from initDevice would now be misread as failure, but no such build has been observed.

7.5 Optional hardening (proposed, NOT yet applied)

The bridge caches its session cookie forever; a BioMiniWebAgent restart can invalidate it. Resetting the cookie when initDevice fails makes recovery automatic:

```
  if (!scanner) {
    cookie = null; // force a fresh BioMini session on the next attempt
    console.error(...);
  }
```

8. Frontend Flow

EnrollForm.tsx checks scanner status on mount, lets the user capture a template, then submits personal data and the base64 template to POST /api/people. It blocks saving when there is no successful scan or while scanning/saving. On success it clears the form and increments refreshKey so PeopleList reloads.

```
// web/src/scanner.ts: central capture adapter
const AGENT_URL = import.meta.env.VITE_AGENT_URL ?? "http://localhost:8443";
const MOCK_FINGER = import.meta.env.VITE_MOCK_FINGER ?? "";
export async function captureFingerprint(timeoutMs = 20000) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), timeoutMs);
  try {
    const query = MOCK_FINGER ? `?finger=${encodeURIComponent(MOCK_FINGER)}` : "";
    const res = await fetch(`${AGENT_URL}/api/capture${query}`, {
      method: "POST", signal: controller.signal,
    });
    if (!res.ok) throw new Error(`Scanner returned ${res.status}`);
    const data = await res.json();
    if (!data.template) throw new Error("No fingerprint returned");
    return { templateBase64: data.template, format: data.format ?? "SUPREMA" };
  } finally {
    clearTimeout(timer);
  }
}
```

PeopleList.tsx calls GET /api/people and renders name, national ID, phone, format and created date. It never receives fingerprint bytes from the API.

9. Prisma Data Model

```
generator client { provider = "prisma-client-js" }
// SQLite keeps setup to zero. For PostgreSQL change provider and DATABASE_URL only.
datasource db { provider = "sqlite"; url = env("DATABASE_URL") }
model Person {
  id          Int       @id @default(autoincrement())
  fullName    String
  nationalId  String?   @unique
  phone       String?
  // AES-256-GCM blob: iv(12) + authTag(16) + ciphertext. Never the raw template.
  fingerprint Bytes
  fpFormat    String
  createdAt   DateTime  @default(now())
}
```

The SQLite URL in the example is file:./dev.db relative to the Prisma schema directory, typically api/prisma/dev.db. Confirm the actual working directory and Prisma resolution if a database appears in an unexpected location.

10. Known Incomplete Areas / High-Value Debugging Leads

Rev 1.1 updates in brackets; everything else carries over from rev 1.0.

- **[FIXED rev 1.1]** BioMini retValue string quirk (§7.3/§7.4): bridge now normalises with retOk(); four strict comparisons replaced. Verification of live capture still pending (§14.5).
- **[FIXED rev 1.1]** api/tsconfig.json had "include": ["src", "../web/src/PeopleList.tsx"], violating rootDir (TS6059) and breaking npm run typecheck before the agent/web checks ran. Now ["src"] only; web sources are checked by the web workspace's own typecheck.

- **[OPEN]** initDevice on the session machine reports DeviceType:"Error" and a mojibake ScannerName; harmless so far (ScannerCount 1, retString Success) but re-check if capture fails with a vendor retString after the patch. The mojibake is an encoding artifact and only affects the display-only model field.
- Real biometric matching is incomplete: real agent /api/match responds 501. With MATCHER=agent, duplicate checking fails with 503 at POST /api/people rather than saving. Use MATCHER=mock until UFMatcher integration exists.
- Mock matching is not biometric matching: it only detects byte-identical templates.
- Vite proxy is a critical dependency: verify web/vite.config.ts forwards /api to http://localhost:3000.
- BioMini compatibility is fragile in more ways than TLS: endpoint names, query parameters, response field names *and field types* can all differ between vendor builds (the type difference is what broke rev 1.0).
- Scanner status and actual capture may disagree; biominiStatus is deliberately lenient about a missing retValue (rev 1.1) — trust capture-path errors, which carry the vendor retString.
- Service liveness: a sudden curl: (7)/ERR_CONNECTION_REFUSED on 8443 usually means the agent process (or the whole concurrently-launched stack) exited; also rule out IPv6-only binding by testing http://localhost:8443 and http://[::1]:8443 against http://127.0.0.1:8443.
- Single-process enrollment queue only: serialize() prevents races inside one API process, not across instances.
- Database uniqueness is the final national-ID guard: Prisma P2002 is caught as 409.
- Encryption key must exist at startup: assertKey() throws before the server starts if FP_ENC_KEY is bad.
- Mock scanner policy: ALLOW_MOCK_SCANNER=false rejects records whose fpFormat is MOCK.
- Privacy/security: no authentication on these routes; loopback binding limits but does not replace access control.
- Input hardening: base64 regex + minimum 32 decoded bytes; consider stricter canonical base64 and rate/size limits if ever exposed beyond localhost. Async error handling: ensure rejected promises are logged with context, without logging biometric data or secrets.

11. Troubleshooting Checklist

- Run npm run typecheck from the repository root and capture the complete output (should be green since rev 1.1).
- Check each service independently: GET http://127.0.0.1:3000/api/health, GET http://127.0.0.1:8443/api/status, and GET http://127.0.0.1:8443/api/debug in real scanner mode.
- Interpret /api/debug: createSessionID 200 ⇒ vendor service reachable; initDevice ScannerCount/retValue ⇒ device visibility (judge success by Number(retValue) === 0, not === 0); getScannerStatus shape varies by build (§7.3).
- If a port probe fails with curl: (7): list listeners with netstat -ano | findstr /R /C:":8443" /C:":3000" /C:":5173" /C:":8083" and map PIDs with Get-Process -Id <PID>; a stale holder can be stopped with Stop-Process -Id <PID> -Force.
- Check that ports 3000, 5173, 8443 and 8083 are not occupied by stale processes.
- Inspect browser DevTools Network: note exact URL, status code, response body, and whether the failed request went to Vite or the API. A JSON body with a Persian sentence came from the agent or API; a bare network error means the service was down.
- Inspect terminal logs separately for agent and API. Redact tokens, keys, personal details and template payloads.
- For enrollment failures, separate the stages: scanner capture, frontend request, API schema validation, national-ID uniqueness, matcher call, encryption/write, read-back/decryption integrity.
- For real scans, verify BioMiniWebAgent is running, the device is detected, its TLS endpoint responds, and the session/cookie and response-field contract matches the installed vendor version.
- Do not change multiple layers at once. Reproduce one failure, form a hypothesis, make one minimal change, rerun.

12. Prompt to Give Another AI

Copy the following prompt into the other AI along with this PDF when starting a debugging session:

```
You are acting as a senior TypeScript/Node.js debugging engineer. Read this project handoff carefully
and use it
as the architecture reference for a local fingerprint enrollment application.
When I report a bug:
1. First identify the most likely layer: React/Vite frontend, Express API, Prisma/SQLite, scanner
agent,
   BioMiniWebAgent/TLS, encryption, or matcher integration.
2. Ask for only the minimum missing evidence needed to diagnose it (exact error, stack trace, HTTP
status/body,
   relevant sanitized config, or source snippet).
3. Explain the root cause in plain English and distinguish confirmed facts from hypotheses.
4. Give a step-by-step reproduction and diagnosis plan, with exact Windows PowerShell commands when
useful.
5. Propose the smallest safe code change. Show the file path and a focused patch or complete
replacement
   function, not vague advice.
6. Explain why the fix works and what regression it could introduce.
7. Give a verification test, including expected output and a negative/edge-case test.
8. Preserve the current architecture unless a redesign is necessary. Do not invent undocumented
Suprema SDK APIs
   or claim real biometric matching works until UFMatcher integration is implemented and tested.
9. Never ask me to paste .env secrets, FP_ENC_KEY, real personal information, database contents, or
fingerprint
   templates. Request redacted logs only.
10. Remember the known limitations: the mock matcher only compares exact bytes; the real agent
/api/match returns
    501 in AGENT_MODE=biomini; since rev 1.1 the bridge normalises vendor retValue strings via retOk()
— never
    reintroduce strict === 0 comparisons on vendor replies. Treat these as facts, not regressions.
Project components:
- web: React 18 + TypeScript + Vite, port 5173.
- api: Express + Zod + Prisma + SQLite, default port 3000.
- agent: local Express scanner bridge, default port 8443 (biomini.ts patched in rev 1.1).
- vendor service: BioMiniWebAgent HTTPS at 127.0.0.1:8083 (returns retValue as string in the installed
build).
- Templates are encrypted with AES-256-GCM before storage. GET /api/people deliberately omits
templates.
- Key source files: api/src/server.ts, api/src/crypto.ts, api/src/matcher.ts, api/src/normalize.ts,
  agent/src/server.ts, agent/src/biomini.ts, web/src/scanner.ts, web/src/EnrollForm.tsx,
web/src/PeopleList.tsx,
  api/prisma/schema.prisma.
Do not assume a bug is fixed just because code looks plausible. Ask me to run the verification steps
and evaluate
the actual result before concluding.
```

13. Data Safety Note

The supplied archive includes files named .env in addition to .env.example templates. Their contents have intentionally not been reproduced in this document. If those real environment files contain credentials, API keys or

encryption keys and the archive has been shared, rotate any exposed credentials. Never publish FP_ENC_KEY or include it in a bug report. The included dev database may also contain biometric test data; treat it as sensitive. The /api/debug evidence quoted in §7.3 and §14 contains no secrets and no template data.

Scope limitation: this brief was generated by inspecting source files in the supplied archive plus one live debugging session on the user's Windows machine. It does not include a successful real-scanner capture after the rev 1.1 patch, physical scanner validation, or verification of the installed Suprema vendor version. Any diagnosis should be confirmed against actual logs and repeatable tests.

14. Debugging Session Log — 2026-10-13 (502/503 investigation)

14.1 Reported symptoms

- Clicking Scan in the UI returned HTTP 502 from the agent with the Persian body quoted in §7.3; 503s were seen intermittently (matches the connection-level BiominiError while BioMiniWebAgent restarts or times out).
- Later the same day, curl http://127.0.0.1:8443/... returned curl: (7) connection refused: the agent process was down (whole-stack exit via concurrently, closed window, or crash). After restarting with npm run dev the port answered again.
- Topology: frontend, API, agent, BioMiniWebAgent.exe and the USB scanner all on one Windows machine, opened via http://localhost:5173.

14.2 Evidence

- GET /api/status → 200 {"mode":"device","scannerConnected":false} — agent alive, vendor reachable, device "missing".
- GET /api/debug → the JSON quoted in §7.3: createSessionID OK; initDevice ScannerCount 1, retString "Success", retValue "0" (string); DeviceType "Error"; mojibake ScannerName; getScannerStatus {"sessionId":"0"} with no retValue.
- Conclusion: the device IS detected by the vendor service; the bridge discarded it because "0" === 0 is false.

14.3 Root cause

Primary (confirmed): strict-equality checks on a vendor field that the installed BioMiniWebAgent build returns as a string, in initScanner() (and latent in doCapture()/biominiStatus()). Secondary (confirmed): the agent process had exited at least once, which produces bare connection refusals rather than JSON errors. Not involved: frontend, Vite proxy, API, Prisma/SQLite, encryption.

14.4 Changes applied on the session machine

- agent/src/biomini.ts: added retOk() helper; initScanner uses retOk(r); biominiStatus fails only on an explicit non-zero retValue; doCapture uses retOk(cap) and retOk(tpl). Exact code in §7.4.
- api/tsconfig.json: "include": ["src", "../web/src/PeopleList.tsx"] → ["src"] (fixes TS6059 rootDir error).
- Proposed but not applied: cookie reset on initDevice failure (§7.5).

14.5 Verification status

- npm run typecheck: api workspace error resolved by the tsconfig fix; full three-workspace green to be confirmed.
- PENDING: after agent restart, GET /api/status should return scannerConnected:true; POST /api/capture with a finger on the sensor should return 200 {"template":"…","format":"SUPREMA"} (never paste templates).
- PENDING negative test: unplug scanner → /api/status reports scannerConnected:false without a crash → replug → true.
- PENDING end-to-end enrollment in the browser; with MATCHER=agent the save step still 503s (known 501 /api/match) — use MATCHER=mock until UFMatcher exists.

14.6 Open items

- DeviceType:"Error" / mojibake ScannerName: confirm harmless once a live capture succeeds; otherwise compare the installed vendor BiominiWebAgent.js endpoint contract and response encoding.

- Real biometric matching (UFMatcher) and the 501 /api/match endpoint remain unimplemented.
- Session-cookie hardening (§7.5) and, if the app ever leaves localhost, authentication, rate limiting and secrets management (§10).